

0.10. Tout l'alphabet, avec poserS

poserS passe à la ligne tout seul : une seule boucle pour les 26 lettres.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : `poserS`. Le reste (`for`, `sizeof`, `ALPHABET[i]`) vient des chapitres précédents : c'est le programme du 0.9, en plus simple.

`sizeof(ALPHABET)` veut dire « la taille de ALPHABET » : le nombre de cases du tableau, **26**. On ne l'écrit plus à la main : si le tableau changeait de taille, la boucle suivrait toute seule. `i < sizeof(ALPHABET)` s'arrête donc après `i = 25`, la dernière lettre.

`poserS(colonne, ligne, tuile)` fait partie de la console, comme `ALPHABET` : on s'en sert sans rien déclarer. C'est `poser()`, mais qui **passe à la ligne tout seul** : une colonne trop grande (20 ou plus) repart à gauche, une ligne plus bas.

On lui donne donc simplement `i` comme colonne, sur la ligne 0. Tour par tour : **i = 0** → A en (0, 0) · **i = 19** → T en (19, 0), la ligne est pleine · **i = 20** → la colonne 20 n'existe pas : `poserS` pose U en (0, 1) · **i = 25** → Z en (5, 1).

Comment `poserS` trouve la place, pour les curieux : la ligne est le nombre de rangées de 20 déjà pleines ($20 / 20 = 1$), et la colonne ce qui reste une fois ces rangées retirées ($25 - 20 = 5$). En C++ : `i / 20` et `i % 20`. `poserS` fait ce calcul à notre place.

Le calcul du 0.9 a disparu du programme : `poserS` le fait à notre place.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <poserS>` → [0.9.3. La fonction poserS\(\) — poser, et passer à la ligne](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **946 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
main()         72 o ← démarrage — ta fonction
VBlank         8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AdresseCase    10 o ← poser() — trouve l'adresse d'une case de l'écran
Diviser        17 o ← poser() — divise — le processeur ne sait pas le faire
Reste          10 o ← poser() — le reste d'une division
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles — la police entière
Lettres gravées : toutes (poserS() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
```

Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 1    47 o sur place, dans main() + AdresseCase (10 o), Diviser (17 o), Reste (10 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
poserS() × 1   0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <poserS>      // pose une tuile, et passe à la ligne au bord
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

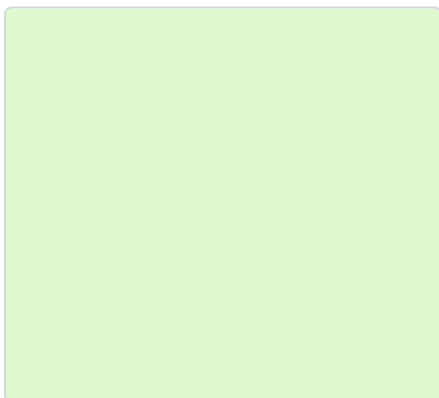
```
int main() {
    // for : répète le bloc entre { } ; i part de 0 et augmente de 1 à chaque tour.
    // sizeof(ALPHABET) = 26, la taille du tableau : 26 tours, i va de 0 à 25.
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {

        // poserS(colonne, ligne, tuile) : comme poser(), mais passe à la ligne tout seul.
        //   i                = la colonne : de 0 à 19 elle tient sur la ligne 0 ;
        //                   à partir de 20, poserS repart à gauche une ligne plus bas
        //                   (i = 20 → U en (0, 1), i = 25 → Z en (5, 1))
        //   0                = la ligne de départ
        //   ALPHABET[i] = la lettre n° i : A pour i = 0, Z pour i = 25
        poserS(i, 0, ALPHABET[i]);
    }

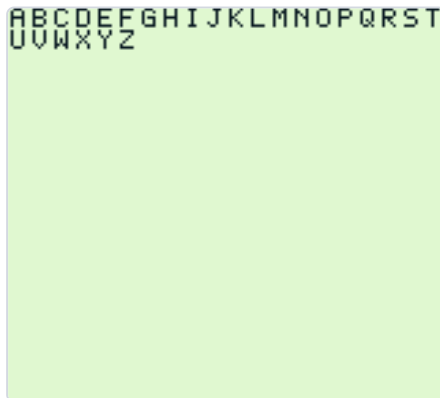
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur la ligne 0, U à Z en dessous, comme au 0.8.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 0 : A à T

✓ la ligne 1 : U à Z

Ensuite : **0.10.1. Tout l'alphabet, avec poserS — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++